

10/1 AFFINE GAP ALGORITHM

Inputs: $X, Y, \alpha, \mu, \gamma, \tau$

- X, Y = strings of length m, n with characters indexed by i, j
- α = match score
- μ = mismatch penalty
- γ = gap opening penalty
- τ = gap extension penalty (single-letter gap penalty)

Output: an alignment which maximizes the following score:
 $\alpha(\# \text{matches}) - \mu(\# \text{mismatches}) - \gamma(\# \text{gap clusters}) - \tau(\# \text{single-letter gaps})$

We have some auxiliary data structures:

Matrices V, G, E, F

↳ V = best-score matrix

↳ G = match-mismatch matrix

↳ E = X -gap matrix

↳ F = Y -gap matrix

We can use an example to see how this works.

$X = A T C G G C$	$\alpha = 2$	$\gamma = 2$
$Y = A G C$	$\mu = 1$	$\tau = 1$

this example will go through every branch in the program to help us see how it works.

G A T C G G C

A -3

G -4

C -5

check PPT for matrices

1) row initialization

2) column initialization

- The first loop takes m iterations

- The second loop takes n iterations

- The third loop takes $m \times n$ iterations

so the overall
time complexity
is quadratic

In V , we put G/E/F depending on which direction we came from. (which direction we got the max from)

↳ This helps with backtracking from the bottom right

↳ G = diagonal // recurse on $V(i-1, j-1)$

↳ E = gap in X // recurse on $V(i, j-1)$

↳ F = gap in Y // recurse on $V(i-1, j)$

★ The concepts of this algorithm are universal - thinking about Shazam.

If we're dealing with a global alignment matrix with a scoring scheme that has:

- a) negative numbers
- b) only positive numbers

Both will give us the same results for global alignment. But, if we were dealing with local alignment, we would have to be more selective with the scoring scheme * BIOLOGICAL RELEVANCE

Random biosequences

$$P(A) = 4/12$$

$$P(B) = 5/12$$

$$P(C) = 3/12$$

A	A	B
B	A	C
B	B	C
C	B	A

- Global alignment of these random sequences of length N 
(length of the optimal alignment $O(N)$)

- Local alignment of these random sequences of length N 
(length of the optimal alignment $O(\log N)$) * how is it stopping at length $O(\log N)$?

THE SCORING SCHEME

Karlin - Aetschul Statistics (1990)

(only for ungapped alignment)

5 AXIOMS:

- 1) Scoring matrix needs to have at least 1 positive score
- 2) The expected score of the scoring matrix needs to be negative
- 3) The letters of the random sequences are independent and identically distributed
- 4) Sequences are infinitely long
- 5) Alignments do not contain gaps

$\{ =$ subsequences maximal in their scoring such that extending will decrease the total score

database search of size n and a query seq. of size m

Karlin - Aetschul equation: $E = K m n e^{-\lambda S}$

$Q_{size m}$ 

$\rightarrow E = \#$ alignments expected by chance for alignments of score S .

Dayhoff LOD Score: log odds ratio

$\rightarrow LOD(\text{pair of amino acids}) = \log_2 \left(\frac{\text{observed frequency}}{\text{expected frequency}} \right)$

$S_{ij} = \frac{q_{ij}}{p_i p_j}$ ($p_i, p_j =$ frequencies of amino acids i & j) in the random model

If observed frequency = expected frequency, $LOD = 0$

If $LOD > 0$, then pairing i & j in alignment is common.

If $LOD < 0$, then pairing unlikely.